

PETRI CONSENSUS

A Continuous-Forge Consensus Protocol for
High-Throughput Blockchain Infrastructure

DEMOS NETWORK · INTERNAL PITCH DOCUMENT · SEED STAGE

Named after the petri dish: a controlled environment where independent cultures grow, interact, and produce observable results. Petri Consensus treats each shard as a biological culture — autonomous, self-verifying, and deterministically composed.



EXECUTIVE SUMMARY

The Problem

Current blockchain consensus mechanisms force a fundamental trade-off: either every validator processes every transaction (limiting throughput), or the network fragments into isolated execution environments (breaking composability). BFT-based systems achieve safety but at the cost of per-transaction coordination overhead that grows with validator count. The result is an industry stuck between ~1,000 TPS with full security or opaque scaling solutions that reintroduce centralization.

The Insight

Most transactions in a well-functioning blockchain do not actually conflict. BFT consensus is essential for safety, but using it as the primary execution engine is wasteful — it should be an **exception handler**, not the main loop. If nodes can independently execute non-conflicting transactions and only invoke heavyweight consensus when disagreements arise, throughput scales dramatically while maintaining full Byzantine fault tolerance.

Petri Consensus

Petri Consensus implements this insight through a three-phase architecture: **instant cryptographic validation** at the RPC layer, **continuous-forge execution** within rotating 10-node shards that sync and verify state deltas every 1–2 seconds, and **BFT arbitration only for conflicting transactions** at the 10-second block boundary. The result: the vast majority of transactions reach finality without ever triggering BFT, while the small minority of conflicts are resolved with full Byzantine safety guarantees.

5,000–15,000

Target TPS per shard
(testnet milestone)

1–2s

Soft finality
(pre-approval)

10s

Hard finality
(block confirmation)

PROTOCOL MECHANISM

Petri Consensus operates in three temporal phases, each optimized for its specific role in the transaction lifecycle.

PHASE 1 — INSTANT VALIDATION

01

Transaction Submission

Client submits a signed transaction to any RPC endpoint in the network. The RPC node is stateless and serves as a cryptographic gatekeeper.

02

Cryptographic Validation & Routing

The RPC node verifies the transaction signature, checks format validity, and deterministically routes to two members of the assigned shard. Shard assignment is derived from the transaction's address space.

03

Shard-Level Verification & Classification

Each receiving shard member independently verifies cryptography and classifies the transaction: **read-only / non-state-changing** transactions are immediately marked as **PRE-APPROVED**. **State-changing** transactions are executed speculatively, producing a state delta, and marked as **TO-APPROVE**.

PHASE 2 — CONTINUOUS FORGE (1–2 second cycle)

04

Mempool Synchronization

Every 1–2 seconds, shard members synchronize their local mempools using the Continuous-Forge merge algorithm. This produces a deterministic ordering of all pending transactions across the shard.

05

Parallel Re-execution & Delta Verification

Each shard member re-executes all TO-APPROVE transactions against the merged, ordered mempool and produces a state delta. Deltas are compared across the shard. If $\geq \frac{7}{10}$ members agree (BFT threshold: $\lceil \frac{2n}{3} + 1 \rceil$), the transaction is promoted to **PRE-APPROVED**.

06

Conflict Detection

Transactions that fail to reach delta agreement are flagged as **PROBLEMATIC**. These are quarantined from the happy path and deferred to Phase 3 for BFT resolution.

PHASE 3 — BLOCK FINALIZATION (10-second boundary)

07

Block Compilation

At the 10-second mark, shard members compile all PRE-APPROVED transactions into a candidate block. Transaction ordering is deterministic, derived from the merge algorithm's output. This is the happy path — for most blocks, this step completes without requiring any additional consensus rounds.

08

BFT Arbitration (Exception Path)

PROBLEMATIC transactions enter a standard BFT round ($\lceil 2n/3 \rceil + 1 = 7/10$ agreement). If consensus is reached, the transaction is included in the block. If not, the transaction is **rejected** — the chain never stalls. This fail-safe design means BFT latency only affects conflicting transactions, not overall throughput.

SHARD ROTATION MECHANISM

Shard composition is **deterministic, rotating, and democratic**. Every block, shard membership is recalculated using a PRNG seeded with the hash of the previous block. This guarantees:

● Unpredictability	No party can predict shard composition more than one block ahead, preventing targeted collusion.
● Determinism	Every node independently computes identical shard assignments from the same block hash — no coordination needed.
● Democratic Rotation	Over time, every validator participates in every shard with uniform probability, preventing power concentration.
● Verifiability	Any observer can verify that shard assignments are correct by rerunning the PRNG with the public block hash.

Shard size: 10 validators — chosen to balance BFT quorum efficiency (7/10 threshold), network overhead (manageable sync every 1–2s), and security (tolerates up to 3 Byzantine actors per shard per block).

TRANSACTION LIFECYCLE

The diagram below traces a transaction from client submission through final block inclusion. The green path represents the optimistic case (majority of transactions). The red dashed path represents the exception handling for conflicting state transitions.

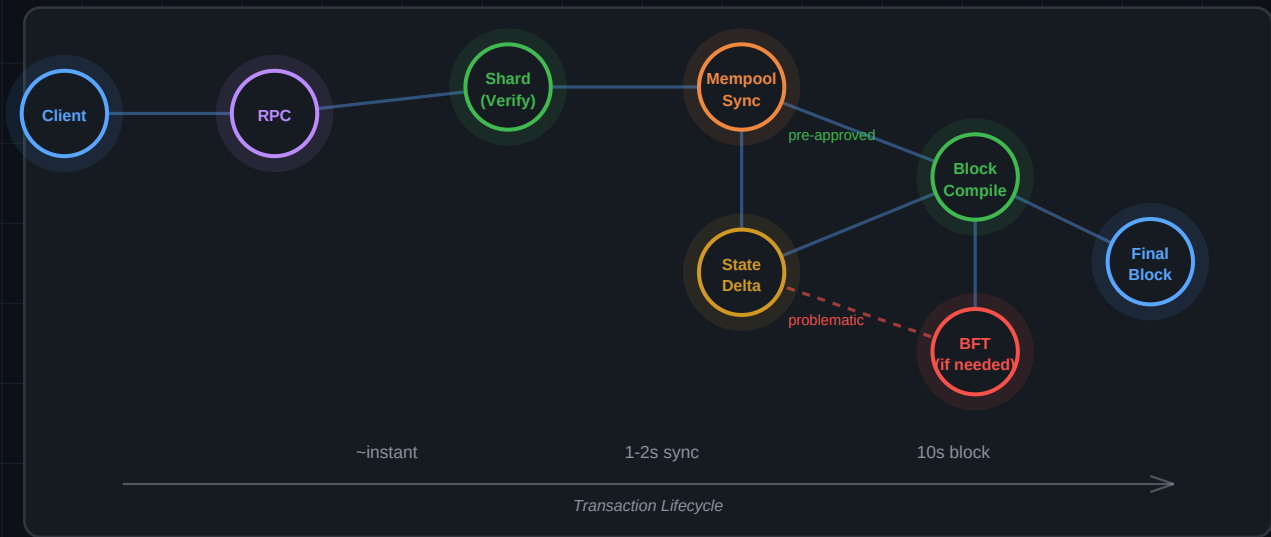


Fig. 1 — Petri Consensus transaction lifecycle. Happy path (solid lines) vs. exception path (dashed red).

SECURITY PROPERTIES

Byzantine Fault Tolerance	Each shard tolerates up to 3 malicious validators ($f < n/3$ where $n=10$). Both the continuous-forge state verification and the BFT arbitration phase use the same $\lceil 2n/3 \rceil + 1$ threshold, ensuring consistent safety guarantees.
Anti-Collusion via Rotation	Deterministic shard rotation makes it economically infeasible to coordinate attacks: an adversary would need to control ≥ 4 of 10 randomly selected validators every block, with assignments changing every 10 seconds.
Liveness Guarantee	The chain never stalls. Conflicting transactions are rejected rather than retried, ensuring block production continues on schedule regardless of Byzantine behavior within the shard.
Dual Finality	Soft finality at 1–2 seconds (pre-approval) enables responsive UX for applications, while hard finality at 10 seconds (block inclusion) provides settlement-grade certainty.

PERFORMANCE TARGETS

Performance projections are based on 10-node shard architecture with 1–2 second sync cycles and 10-second block times. These are conservative testnet targets — mainnet optimizations (parallel shard execution, adaptive block sizing, pipelined verification) can significantly increase throughput.

5,000–15,000	10x+
TPS per shard (testnet target)	Horizontal scaling with additional shards

COMPETITIVE LANDSCAPE

	Ethereum 2.0	Solana	Sui / Aptos	Cosmos Zones	Demos (Petri)
Consensus	Gasper (LMD+Casper)	Tower BFT + PoH	Narwhal & Bullshark/Tusk	Tendermint BFT	Continuous-Forge + BFT
TPS	~30–100	~4,000 (theoretical 65k)	~5,000–10,000	~1,000 per zone	5,000–15,000 per shard
Finality	~12 min (epoch)	~0.4s (slot)	~2–3s	~6s	1–2s soft 10s hard
BFT Role	Primary engine	Primary engine	DAG-based ordering	Primary engine	Exception handler only
Validator Load	Full chain processing	Full chain processing	Object-level parallel	Per-zone processing	Shard-scoped (10 nodes)

Demos' key differentiator: BFT as exception handler, not primary engine. This architecture decouples throughput from consensus overhead — as shard count grows, the network scales horizontally without increasing per-validator load.

PETRI CONSENSUS

Where BFT becomes the exception, not the rule.



SPEED

1–2s soft finality 10s hard finality
5,000–15,000 TPS/shard



SECURITY

Full BFT safety Rotating 10-node
shards 3 Byzantine fault tolerance



SCALABILITY

Horizontal shard scaling Constant
validator load Democratic rotation

Demos Network · Unified Identity · Cross-Chain Interoperability

demos.sh